

Sistema de E-commerce Distribuído com AWS (LocalStack) e Terraform

Documento de Definição – Trabalho Final

FURB - Universidade Regional de Blumenau

Disciplina: Sistemas Distribuídos | Prof. Gabriel Castellani de Oliveira

Alunos: Caio Dalagnoli Dranka e Vinicius Muller

1. Descrição do Projeto

O projeto consiste em um sistema distribuído de e-commerce com processamento assíncrono de pedidos. A solução evolui um trabalho anterior baseado em RabbitMQ para uma arquitetura cloud-native utilizando serviços AWS emulados via LocalStack, provisionados com Terraform e implementados em Python.

O sistema permite que um cliente faça um pedido via API REST, que é então processado de forma assíncrona por múltiplos serviços independentes: pagamento, estoque, nota fiscal, logística e notificação. Cada etapa persiste seu estado em banco de dados e consome mensagens de filas SQS.

2. Requisitos Atendidos

Requisito	Tecnologia
Banco de dados (não SQLite)	PostgreSQL via Amazon RDS (LocalStack)
Mensageria	Amazon SQS (LocalStack) + boto3
Integração com API externa	ViaCEP (validação de endereço)
Mínimo dois serviços	6 serviços Python independentes
Uso de nuvem	LocalStack (emulador AWS) + Terraform

3. Arquitetura dos Componentes

A arquitetura é composta pelos seguintes componentes principais:

3.1 Serviço de Pedidos (Produtor)

API REST implementada com FastAPI (Python). Responsável por:

- Receber pedidos via HTTP POST /pedidos
- Validar o endereço de entrega consultando a API externa ViaCEP

- Persistir o pedido no banco de dados PostgreSQL com status PENDENTE
- Publicar mensagem na fila SQS para processamento assíncrono

3.2 Broker de Mensagens (Amazon SQS via LocalStack)

Substitui o RabbitMQ do projeto anterior. Filas provisionadas via Terraform:

- sqs-pedidos-pagamento
- sqs-pedidos-estoque
- sqs-pedidos-fiscal
- sqs-pedidos-logistica
- sqs-pedidos-notificacao
- sqs-dead-letter (DLQ para falhas)

3.3 Consumidores (Serviços Python)

Serviços independentes, cada um rodando em processo separado:

- pagamento.py – Valida pagamento (simulado), atualiza status no PostgreSQL
- estoque.py – Reserva produto, atualiza status no PostgreSQL
- fiscal.py – Gera nota fiscal, persiste no banco
- logistica.py – Agenda entrega, persiste no banco
- notificacao.py – Envia notificação (simulada) ao cliente

3.4 Banco de Dados (PostgreSQL)

Instância PostgreSQL (Amazon RDS emulado via LocalStack ou container Docker). Tabelas principais:

- pedidos – id, cliente, produto, total, status, created_at
- eventos_pedido – id, pedido_id, servico, status, mensagem, created_at

3.5 Infraestrutura como Código (Terraform)

Terraform provisionará no LocalStack:

- Filas SQS (incluindo DLQ e redrive policy)
- Bucket S3 (armazenamento de notas fiscais em JSON)
- Parâmetros no SSM Parameter Store (configurações dos serviços)

3.6 API Externa – ViaCEP

Utilizada pelo Serviço de Pedidos para validar o CEP informado pelo cliente antes de aceitar o pedido. Retorna logradouro, bairro, cidade e UF. Não requer autenticação.

4. Tecnologias Utilizadas

Categoria	Tecnologia
Linguagem	Python 3.11+
API REST	FastAPI
Mensageria	Amazon SQS (LocalStack)
Banco de Dados	PostgreSQL
Nuvem (emulada)	LocalStack
Infra como Código	Terraform
API Externa	ViaCEP
Containerização	Docker / docker-compose

5. Diagrama

